

Consensus Guide

Nakamoto consensus: the double-spend race, the arithmetic of confirmations, and the deployed most-work rule

m@rek.onl

Abstract

Assuming the *Math Guide*'s probability foundations and the hash and commitment foundations of the *Crypto Guide*, this volume of *The Zcash Arboretum* constructs Nakamoto consensus from first principles and quantifies the security it buys. It states the block tree and the most-work rule as the protocol specification defines them and as the deployed zebra implements them; builds, on the *Math Guide*'s foundations, the race-specific probability the analysis needs (memoryless clocks, the biased random walk, the negative binomial law); develops the exact first-seen-model double-spending analysis of Rosenfeld's *Analysis of hashrate-based double-spending* — the catch-up theorem, the confirmation formula, and the economics of the attack — with every derived number recomputed by script and the results instantiated for Zcash's proposed 25-second blocks; explains the draft NU7 transition to 25-second blocks and halving-preserving reissuance; and explains the protocol and implementation safety rails (difficulty adjustment, reorg windows, coinbase maturity, wallet confirmation policy) that frame the model's assumptions. Where the specification, the paper, and the deployed nodes disagree, the divergence is stated.

Contents

1	Scope, sources, and the two double-spends	3
1.1	Sources and their standing	3
2	The block tree and the most-work rule	3
2.1	Work, and the best chain	4
2.2	The attack	4
3	A probability toolkit	5
3.1	Memoryless clocks and the attribution sequence	5
3.2	The negative binomial law	6
4	Playing catch-up	7
5	Waiting for confirmations	9
6	What the numbers say	11
6.1	Zcash time: 25-second blocks	12
7	The economics of the attack	13
7.1	Reward-normalised exposure	14
8	NU7: 25-second blocks and bounded shielded work	15
8.1	Chain selection and the tie-break	15
8.2	Difficulty in motion	15
8.3	Equihash and the memoryless model	16
8.4	Reorg rails, maturity, and the finality floor	17
8.5	Wallet confirmation policy	17
8.6	A block budget for shielded work	17
8.7	Scheduled issuance with shorter blocks	18
8.8	The NSM reserve and delayed reissuance	18
9	Layer boundary	19

1 Scope, sources, and the two double-spends

The protocol layers above this one take a working ledger for granted. This volume documents the mechanism that selects that ledger: Nakamoto consensus — proof of work, the most-work rule, and the probabilistic finality they purchase. The treatment is quantitative throughout. Its spine is Meni Rosenfeld’s *Analysis of hashrate-based double-spending* (11 December 2012; latest version 12 February 2014; arXiv:1402.2009), which gives an exact counterpart to the approximate analysis in Satoshi Nakamoto’s whitepaper; we reprove its results in full, recompute its tables by script, and instantiate its conclusions for the proposed NU7 parameters.

Two distinct attacks are both called “double-spending”, and this volume treats exactly one of them. Two spends of the same shielded note reveal the same nullifier, and the protocol specification’s §“Nullifier Sets” forbids a repeated nullifier within a transaction or valid block chain; such a same-chain attempt is therefore invalid under the consensus rules. What consensus must prevent is the chain-level attack: paying a merchant on the chain everyone sees, then replacing that chain wholesale with another in which the payment never happened. No zero-knowledge proof (*Crypto Guide*, §“Interactive proofs, zero knowledge, and SNARKs”) rules this out; under the model developed here, the proof-of-work race gives the attack its probabilistic bound.

The quantitative argument assumes the *Math Guide*’s probability section (§“Probability, asymptotics, and computation”), which constructs probability from its definition — finite spaces, conditioning, independence, random variables, expectation — and extends it to the three rungs used here: countable additivity, continuity along monotone events, and density-defined laws (§“Beyond finite spaces: countable additivity, limits, and densities”). The block and anchor vocabulary uses the *Crypto Guide*’s hash and commitment sections. On that base, Section 3 constructs the race-specific instruments no lower volume owns — memoryless clocks, random walks, the negative binomial law; the algebra used is elementary. Readers wanting only the results can read Sections 2, 6, and 8 and take the theorems on faith.

1.1 Sources and their standing

Rosenfeld’s analysis supplies the probability and economic model, not consensus rules; its two printed tables are reproduced by exact arithmetic before instantiating the model here. The protocol specification and the relevant ZIPs define the protocol.

This volume targets the *draft NU7 rules* as of 23 September 2026: ZIP-259 selects ZIP-218, ZIP-237, and ZIP-2003. Activation heights remain unassigned, and some proposed limits and issuance rules are incomplete. These boundaries are stated where they affect a calculation; no unassigned constant is treated as a deployed rule.

Remark 1.1 (The paper’s own condition). The arXiv v1 PDF of the paper is lightly broken: it contains exactly two citations, both rendered as “[?]” (in the abstract and in §1), and no References section at all. Both plainly intend the Nakamoto whitepaper, and we read them so. The paper numbers only two of its displayed equations — its equation (1) is our Theorem 5.2, its equation (2) our equation (3) — and sets its two model assumptions as a bare numbered list; the *Assumption* environments of Section 4 are ours.

2 The block tree and the most-work rule

Zcash transactions are grouped into blocks, and every block names its parent by including the parent header’s hash (*Crypto Guide*, §“Hash functions and the random oracle model”); the genesis block alone has no parent. Blocks therefore form a tree rooted at genesis, and each root-to-leaf path — each *branch* — is one internally consistent version of history. Branches need not be consistent with one another: one branch can contain a transaction that conflicts with a transaction in a sibling branch, and that freedom is exactly the attack surface this volume quantifies.

2.1 Work, and the best chain

A single coherent history requires a rule every node can apply locally and use to converge once one valid branch is strictly preferred. The rule is not simply “longest”: it weighs each block by the computational effort represented by its header target.

Definition 2.1 (Work of a block). Let `ToTarget` be the conversion from the header’s `nBits` field to its 256-bit target threshold (specification §7.7.4). The *work* of a block is

$$\text{work} = \left\lfloor \frac{2^{256}}{\text{ToTarget}(\text{nBits}) + 1} \right\rfloor$$

(specification §7.7.5, “Definition of Work”). A lower target admits fewer valid headers, so a block meeting it certifies proportionally more expected hashing; work is that certificate made additive.

The deployed Zebra validator computes this quantity with a 256-bit in-word trick: zebra evaluates $(\text{!expanded}.0 / (\text{expanded}.0 + 1)) + 1$; the retired `zcashd` computed the identical expression on `arith_uint256`, with `~` spelling the complement. The form agrees with Definition 2.1 exactly: writing t for the target and $A = 2^{256}$, the complement is $\neg t = A - 1 - t$, and

$$\left\lfloor \frac{A - 1 - t}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A - (t + 1)}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A}{t + 1} \right\rfloor,$$

so the in-word computation equals the out-of-word quotient (checked by script over random targets).

Definition 2.2 (Best valid block chain). Specification §3.3 (“The Block Chain”): a node “sums the work, as defined in §7.7.5, of all blocks in each valid block chain, and considers the valid block chain with greatest total work to be best. To break ties between leaf blocks, a node will prefer the block that it received first.”

Rosenfeld’s paper states the same rule for Bitcoin — “the branch representing the most proof of work”, with first-seen tie-breaking — so the model uses the same selection rule. The shorthand “longest chain” is exact only while difficulty is constant, which is precisely the paper’s Assumption 4.2 and not a property of the deployed chain (Section 8.2). Note also what the rule does *not* say: nothing marks a chain as permanently chosen. A node that learns of a heavier branch switches to it, however deep the fork point — subject only to the node-local rails of Section 8.4. Consensus is a standing competition, not an election with a winner.

Definition 2.3 (Confirmations). A transaction has n *confirmations* if it is included in a block of the best chain and there are n blocks on the path from that block to the chain’s leaf, inclusive. The block containing the transaction is its first confirmation.

Different nodes may briefly disagree on the best chain when two blocks of equal cumulative work arrive in different orders. Absent a concurrent extension of the other branch, a later block makes one branch strictly heavier, and nodes converge on it once that block propagates (Figure 1). Such ties are the benign case. The attack of Section 5 is the malicious one: a fork manufactured in private and released only when it wins.

2.2 The attack

The double-spend attack, as the paper lays it out (§3):

1. broadcast a transaction paying the merchant;

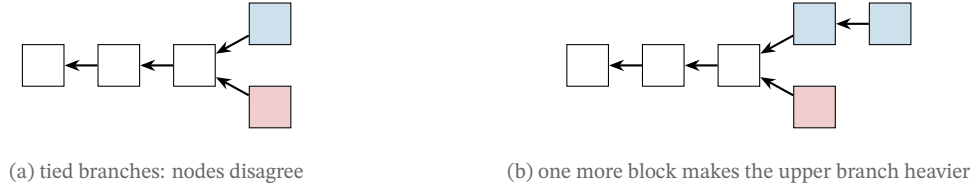


Figure 1: Branch selection. Arrows point from child to parent. In panel (a) two blocks extend the same parent and the branches carry equal work; each node keeps the one it saw first. In panel (b) a new block lands on the upper branch, which now has strictly more cumulative work; during propagation, every node that knows this state selects the upper branch, unless the lower branch has meanwhile been extended. At constant difficulty “more work” and “longer” coincide, which is how the paper, and this volume’s figures, may draw block counts.

2. secretly mine a branch from the last pre-transaction block, containing instead a conflicting transaction paying the attacker himself;
3. wait until the merchant’s transaction has n confirmations and the merchant, satisfied, hands over the goods;
4. in the paper’s first-seen tie model, keep extending the secret branch until it carries more work than the public one, then broadcast it. Every node that sees it switches to it, the payment to the merchant is no longer part of history, and the attacker has both the goods and the coins.

Nothing in step 4 violates any validity rule — the released branch is a perfectly well-formed chain, merely a different one. The only question is probabilistic: with what probability does the secret branch ever get ahead? Answering it exactly in that first-seen model is the business of Sections 4 and 5; the toolkit comes first.

3 A probability toolkit

The analysis ahead needs four tools: memoryless clocks (to model block discovery), a reduction from continuous time to a coin sequence, the negative binomial distribution (to count the attacker’s progress), and one normalisation lemma about races. Their foundations are the *Math Guide*’s: probability spaces, conditioning, independence, random variables, and expectation are constructed there from the definition of probability onward (§“Probability, asymptotics, and computation”), and the extension beyond finite spaces — countable additivity, continuity along monotone events, density-defined laws, and infinite sequences of independent trials — is its §“Beyond finite spaces: countable additivity, limits, and densities”. What no lower volume owns are the four instruments themselves; each is constructed here on that base, only as far as the critical path requires.

3.1 Memoryless clocks and the attribution sequence

Definition 3.1 (Exponential clock). A random variable X is *exponential with rate* $\lambda > 0$ if $\Pr[X > t] = e^{-\lambda t}$ for all $t \geq 0$ — a law defined by the density $\lambda e^{-\lambda t}$ on $t \geq 0$ in the sense of the *Math Guide*’s density definition, with mean $1/\lambda$.

Lemma 3.2 (Memorylessness). An exponential X satisfies $\Pr[X > s + t \mid X > s] = \Pr[X > t]$ for all $s, t \geq 0$: having waited without success teaches nothing about the remaining wait.

Proof. $\Pr[X > s + t \mid X > s] = e^{-\lambda(s+t)}/e^{-\lambda s} = e^{-\lambda t}$. □

Lemma 3.3 (Race of two clocks). *Let X and Y be independent exponentials with rates λ and μ . Then*

$$\Pr[X < Y] = \frac{\lambda}{\lambda + \mu},$$

and the winning time $\min(X, Y)$ is exponential with rate $\lambda + \mu$.

Proof. Integrating over the time at which X fires,

$$\Pr[X < Y] = \int_0^\infty \lambda e^{-\lambda t} \Pr[Y > t] dt = \int_0^\infty \lambda e^{-(\lambda+\mu)t} dt = \frac{\lambda}{\lambda + \mu}.$$

For the minimum, $\Pr[\min(X, Y) > t] = \Pr[X > t]\Pr[Y > t] = e^{-(\lambda+\mu)t}$. Moreover, the joint density that X wins at time t factors as

$$\lambda e^{-(\lambda+\mu)t} = \frac{\lambda}{\lambda + \mu} (\lambda + \mu) e^{-(\lambda+\mu)t},$$

and similarly for Y . Thus the winner is independent of the winning time. $\square$

Proposition 3.4 (The attribution sequence). *Model the honest network and attacker as independent exponential clocks with rates p/T_0 and q/T_0 , restarting when either fires (Section 4). Successive block attributions form an infinite sequence of independent, identically distributed Bernoulli trials in the Math Guide’s sense: attacker with probability q , honest network with probability p , independent of all earlier attributions and all elapsed times.*

Proof. By Lemma 3.3 the first block is the attacker’s with probability $(q/T_0)/(p/T_0 + q/T_0) = q$. When a block is found, the finder’s clock restarts by construction, and the loser’s remaining wait is distributed as a fresh clock by Lemma 3.2; the state after each block is therefore probabilistically identical to the start, independent of the past. The density factorisation in Lemma 3.3 also makes each winner independent of that race’s elapsed time. The claim follows by induction. $\square$

Proposition 3.4 is the paper’s bridge (its §3, with footnote 1 supplying the clock rates) from physical time to combinatorics: since the question “does the attacker ever get ahead?” concerns only the *order* of block discoveries, not their times, every result below is a statement about a p/q coin sequence, and the time constant T_0 disappears from the answers. Where the exponential model itself comes from — and how well Zcash’s Equihash fits it — is a deployment question, taken up in Section 8.3.

3.2 The negative binomial law

Proposition 3.5 (Progress of the loser). *In an attribution sequence with attacker probability $q = 1 - p$, let m be the number of attacker blocks found strictly before the honest network’s n -th block. Then*

$$P(m) = \binom{m+n-1}{m} p^n q^m, \quad m = 0, 1, 2, \dots$$

— the negative binomial distribution.

Proof. The event fixes the first $m+n$ trials exactly this far: the $(m+n)$ -th trial is honest (the n -th honest success), and among the first $m+n-1$ trials exactly m are the attacker’s, in any of $\binom{m+n-1}{m}$ orders. Each such order has probability $p^{n-1} q^m \cdot p$. $\square$

Lemma 3.6 (First-to- n race). *In the same sequence, let H_n be the event that the honest network reaches n blocks before the attacker does, and A_n the reverse. Then H_n and A_n partition the sample space, and*

$$\Pr[H_n] = \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^n q^m, \quad \Pr[A_n] = \sum_{h=0}^{n-1} \binom{h+n-1}{h} q^n p^h, \quad \Pr[H_n] + \Pr[A_n] = 1.$$

Proof. Among the first $2n - 1$ trials one side already has n successes, so the race terminates; a tie is impossible because block counts advance one at a time. The event H_n says the n -th honest block arrives while the attacker holds $m \leq n - 1$; summing Proposition 3.5 over those m gives the first sum, and A_n is the same computation with the roles of p and q swapped. $\square$

Remark 3.7 (From hash trials to exponential clocks). The exponential model is itself a limit, not an axiom. A miner’s work is modelled as a stream of candidate evaluations, each an independent Bernoulli trial with a tiny success probability; the number of trials to the first success is geometric, and a geometric law with small success probability, viewed at the scale of its mean, converges to the exponential of Definition 3.1. The step that matters is *progress-freeness*: a candidate that fails must carry no information usable by the next one. Whether Zcash’s Equihash satisfies this is examined with the deployed parameters in Section 8.3.

4 Playing catch-up

The attacker of Section 2.2 finds himself, at the moment the merchant ships, some number of blocks behind the public chain, and wins if his branch ever overtakes it. This section computes the probability of that event as a function of the deficit; the next section supplies the distribution of the deficit itself.

Both computations run under the paper’s two standing assumptions, stated here nearly verbatim (the environment form is ours, Remark 1.1):

Assumption 4.1 (Constant hashrates). The total hashrate of the honest network and the attacker is constant. Combined they have a hashrate of H , of which pH belongs to the honest network and qH to the attacker, where $0 < p, q < 1$ and $p + q = 1$.

Assumption 4.2 (Constant difficulty). The mining difficulty is constant, and such that with hashrate H the average time to find a block is T_0 .

Under Assumption 4.2 every block carries equal work, so “most work” and “longest” coincide and the race can be scored in block counts. Section 8.2 explains how the deployed chain departs from these assumptions — difficulty adjusts after every block and hashrate drifts — and why the formulas should be read as an equal-difficulty baseline rather than an exact deployment model.

Definition 4.3 (The deficit walk). Let z denote the honest chain’s lead in blocks over the attacker’s branch. By Proposition 3.4, each newly found block moves z up by 1 with probability p (honest block) or down by 1 with probability q (attacker block), independently of the past:

$$z_{i+1} = \begin{cases} z_i + 1 & \text{with probability } p, \\ z_i - 1 & \text{with probability } q. \end{cases}$$

The attack succeeds the moment z reaches -1 : the secret branch is then strictly ahead, and releasing it rewrites history.

Theorem 4.4 (Catch-up probability; Rosenfeld §3). *Let a_z be the probability that the walk of Definition 4.3, started at deficit z , ever reaches -1 . Then*

$$a_z = \min(q/p, 1)^{\max(z+1, 0)} = \begin{cases} 1 & \text{if } z < 0 \text{ or } q \geq p, \\ (q/p)^{z+1} & \text{if } z \geq 0 \text{ and } q < p. \end{cases} \quad (1)$$

Proof. For $z < 0$ the walk has already succeeded, so $a_z = 1$; assume $z \geq 0$. Fix an integer $N > z$ and absorb the walk at both -1 and N ; let $h_N(z)$ be the probability of reaching -1 before N . Conditioning on the first step,

$$h_N(z) = p h_N(z+1) + q h_N(z-1) \quad (0 \leq z \leq N-1), \quad h_N(-1) = 1, \quad h_N(N) = 0,$$

a second-order linear recurrence. The trial solution $h_N(z) = t^z$ satisfies it exactly when t is a root of the characteristic polynomial $pt^2 - t + q = p(t-1)(t-q/p)$.

If $q \neq p$ the roots 1 and q/p are distinct, so $h_N(z) = A + B(q/p)^z$; the boundary conditions give

$$h_N(z) = \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N}.$$

If $q = p$ the root is double and $h_N(z) = A + Bz$, giving $h_N(z) = (N-z)/(N+1)$.

Now let $N \rightarrow \infty$. The events $E_N = \{\text{reach } -1 \text{ before } N\}$ are nondecreasing in N , and their union is the event that the walk ever reaches -1 : a walk that does so does it at a finite time, having visited finitely many states, so E_N holds for every N above the largest state visited. By continuity along monotone events (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”), $a_z = \lim_{N \rightarrow \infty} h_N(z)$. For $q < p$ the term $(q/p)^N$ vanishes and the quotient tends to $(q/p)^{z+1}$; for $q = p$, $(N-z)/(N+1) \rightarrow 1$; for $q > p$, dividing numerator and denominator by $(q/p)^N$ sends both to -1 , so the quotient tends to 1 . $\square$

Corollary 4.5 (Half or more always catches up). *When $q \geq p$, the attacker overtakes the public chain with probability 1 from any finite deficit. No confirmation count defends when the attacker controls at least half of the hashrate.*

Corollary 4.6 (Geometric decay in the deficit). *When $q < p$, each additional block of deficit multiplies the catch-up probability by q/p . Concretely (script-computed): at $q = 10\%$, $a_0 = 1/9 \approx 0.1111$ and $a_5 \approx 1.88 \times 10^{-6}$; at $q = 30\%$, $a_0 = 3/7 \approx 0.4286$ and $a_5 \approx 6.20 \times 10^{-3}$.*

Corollary 4.7 (Blocks, not time). *Equation (1) contains neither T_0 nor any other time scale: the security of a deficit is a function of block counts alone. Two chains with different block spacings but the same q offer identical security per confirmation — and therefore different security per minute (Section 6.1).*

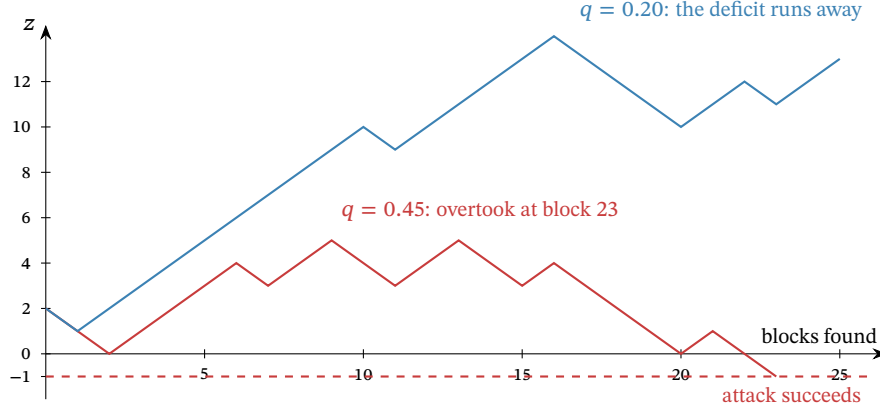


Figure 2: Two runs of the deficit walk from $z = 2$, simulated by script with fixed seeds. At $q = 0.45$ the drift is weak and this run reaches -1 after 23 blocks — the attack succeeds; at $q = 0.20$ the deficit grows roughly linearly, and by Theorem 4.4 the chance of ever returning from deficit 13 is $(1/4)^{14} \approx 3.7 \times 10^{-9}$.

5 Waiting for confirmations

The merchant’s defence is patience: ship only after the paying transaction has n confirmations (Definition 2.3). This section computes exactly how much that patience buys in the paper’s first-seen model. The paper’s model of the attack timeline has one convention worth surfacing before the theorem.

Remark 5.1 (The pre-mined block). The paper assumes the attacker banks one block before the race is scored: “we assume one block was pre-mined by the attacker before commencing the attack” (§4). The attacker forks from the public leaf at the moment the merchant’s transaction is broadcast, and a patient attacker times the broadcast to coincide with a private block he has just found but not announced. At the moment the honest chain reaches n confirmations, the attacker who has found m further blocks therefore holds $m + 1$, and the deficit walk of Definition 4.3 starts at

$$z = n - m - 1.$$

The convention is a modelling choice, not a law: an attacker with no banked block starts at $z = n - m$, and every probability below shrinks accordingly. It is also, as Theorem 5.2 shows, precisely the choice that makes the final formula collapse into a symmetric race.

By Proposition 3.5, the number m of attacker blocks found while the honest network finds its n is negative binomial. Averaging the catch-up probability of Theorem 4.4 over m gives the central result for the specification and Rosenfeld first-seen tie rule. Section 8.1 explains the additional success opportunity supplied by Zebra’s tie-break. The model is not an exact deployed rollback probability.

Theorem 5.2 (Double-spend success probability; Rosenfeld eq. (1)). *Let the merchant wait for $n \geq 1$ confirmations, under Assumptions 4.1 and 4.2 and the convention of Remark 5.1, with a private branch accepted only after it becomes strictly heavier. The probability that the attacker’s branch eventually overtakes the public chain is $r(q, n) = 1$ if $q \geq p$, and otherwise*

$$r(q, n) = 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) = 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n. \quad (2)$$

Proof. Conditioning on m and applying Theorem 4.4 at $z = n - m - 1$,

$$r = \sum_{m=0}^{\infty} P(m) a_{n-m-1} = \underbrace{\sum_{m=0}^{n-1} P(m) \left(\frac{q}{p}\right)^{n-m}}_{\text{behind: must catch up}} + \underbrace{\sum_{m=n}^{\infty} P(m)}_{\text{already ahead}},$$

valid for $q < p$; when $q \geq p$ every $a_{n-m-1} = 1$ and $r = \sum_m P(m) = 1$ immediately. For the first sum, each term transforms exactly:

$$\binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} = \binom{m+n-1}{m} p^m q^n,$$

which by Lemma 3.6 is the probability that the attacker's n -th block arrives while the honest network holds m ; summed over $m \leq n-1$, the first sum is $\Pr[A_n]$, the probability that the attacker wins the first-to- n race. The second sum is $\Pr[m \geq n] = 1 - \Pr[H_n] = \Pr[A_n]$ by the same lemma. Hence $r = 2\Pr[A_n]$, the right-hand form of (2); the middle form follows from $\Pr[A_n] = 1 - \Pr[H_n]$ applied to one of the two copies. The printed upper limit in the paper's equation (1) is n rather than $n-1$; the $m = n$ term is identically zero, so the forms agree. $\square$

Remark 5.3 (The race doubling). The proof exposes a structure the paper's algebra passes through silently: under the one-pre-mined-block convention, the *catch-up half* — the honest chain reaches n first and the attacker later overtakes — has exactly the same probability as the *already-ahead half*, in which the attacker reaches n first. A first-to- n winner may have fallen behind and recovered along the way; neither half asserts otherwise. Thus the double-spend probability is twice the chance of winning a first-to- n race. The identity also absorbs the boundary case: at $q = p$ each race half is exactly $\frac{1}{2}$, giving $r = 1$ with no case split, continuously with the $q > p$ regime. The algebra here is the paper's; reading its two sums as the two halves of one race, and the observation that the pre-mine convention is what makes them equal, are this volume's presentational additions (verified by script over random (q, n) ; the two sums agree to floating-point precision).

Example 5.4 (A full race, exactly). Take $q = 1/5$ (a twenty-percent attacker) and $n = 6$. The deficit starts at $z = 5 - m$, the catch-up factor is $(q/p)^{6-m} = 4^{-(6-m)}$, and every quantity below is an exact rational, printed to six decimals (script-computed):

m	$P(m)$	$a_{5-m} = 4^{-(6-m)}$	product
0	0.262144	0.000244	0.000064
1	0.314573	0.000977	0.000307
2	0.220201	0.003906	0.000860
3	0.117441	0.015625	0.001835
4	0.052848	0.062500	0.003303
5	0.021139	0.250000	0.005285
catch-up half (sum of products)			0.011654
already-ahead half, $\Pr[m \geq 6]$			0.011654
$r(0.2, 6)$			0.023308

The two halves agree to every printed digit — they are equal exactly, by Remark 5.3 — and the closed form (2) evaluates to the same 0.023308. A merchant facing a possible twenty-percent adversary who ships after six confirmations is assigned a 2.33% strict-overtake probability by this model.

Remark 5.5 (Satoshi's approximation). The whitepaper's §11 computes the same quantity with one approximation and two offsetting conventions: the attacker's progress is approximated as Poisson with mean $\lambda = nq/p$ (the law $\Pr[k] = e^{-\lambda} \lambda^k / k!$ on the nonnegative integers); no block is pre-mined, but success is scored at *breakeven* rather than strictly ahead, and those two conventions cancel exactly

— the whitepaper’s catch-up factor for m attacker blocks is $(q/p)^{n-m}$, term for term the factor of Remark 5.1. The Poisson approximation is the entire gap. In each of the following script-computed comparisons it pushes the estimate down: at $q = 10\%$, $n = 6$, the whitepaper’s formula gives 0.0243% against the exact 0.0591% — an understatement by a factor of about 2.4; at $q = 30\%$, $n = 6$: 13.211% against 15.645%; at $q = 10\%$, $n = 10$: 0.0001% against 0.0008%. The direction is not uniform over all (q, n) ; the reason to prefer the negative binomial model is that it is exact under the stated assumptions, not that the approximation is always one-sided.

q	$n=1$	2	3	4	5	6	7	8	9	10
2%	4.000	0.237	0.016	0.0011	$8 \cdot 10^{-5}$			$< 10^{-5}$		
5%	10.000	1.450	0.232	0.039	0.0066	0.0012	$2.1 \cdot 10^{-4}$	$4 \cdot 10^{-5}$	$< 10^{-5}$	
10%	20.000	5.600	1.712	0.546	0.178	0.059	0.020	0.0067	0.0023	0.0008
20%	40.000	20.800	11.584	6.669	3.916	2.331	1.401	0.848	0.516	0.316
30%	60.000	43.200	32.616	25.207	19.762	15.645	12.475	10.003	8.055	6.511
40%	80.000	70.400	63.488	57.958	53.314	49.300	45.769	42.621	39.787	37.218
45%	90.000	85.050	81.375	78.342	75.716	73.375	71.252	69.299	67.487	65.793
48%	96.000	94.003	92.508	91.264	90.177	89.201	88.307	87.478	86.703	85.972
50%					100 for every n					

Table 1: The double-spend success probability $r(q, n)$, in percent, computed by script from Theorem 5.2. On the rows the paper’s Table 1 also prints ($q \in \{2, 10, 20, 30, 40, 48, 50\}\%$), the script reproduces its values cell-for-cell; the 5% and 45% rows are this volume’s additions.

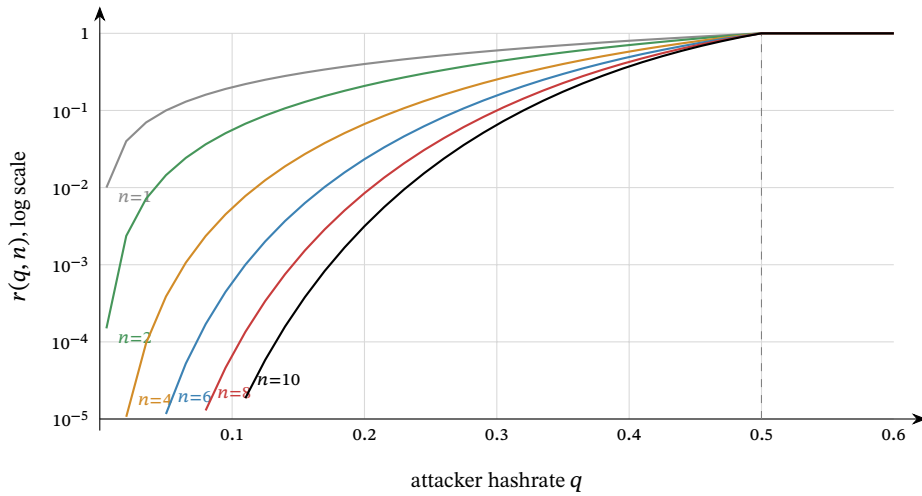


Figure 3: The success probability $r(q, n)$ against the attacker’s hashrate share q , for $n \in \{1, 2, 4, 6, 8, 10\}$ confirmations, logarithmic vertical scale clipped at 10^{-5} ; coordinates computed by script from Theorem 5.2. Every curve reaches 1 at $q = \frac{1}{2}$ (dashed) and stays there: beyond the majority line, confirmations are worthless. Below it, each added confirmation drops r by a roughly constant factor — the curves are near-equally spaced in log scale — and the factor improves as q falls.

6 What the numbers say

Theorem 5.2 compresses the security of Nakamoto consensus into one two-parameter function, and its qualitative lessons are all visible in Table 1 and Figure 3. This section states them precisely, then instantiates the one lesson that is specific to Zcash.

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
2%	1	2	3
5%	2	3	4
8%	2	3	5
10%	2	4	6
15%	3	6	9
20%	4	8	13
25%	5	12	20
30%	9	20	32
35%	15	36	58
40%	34	82	133
45%	135	331	539

Table 2: The least number of confirmations pushing $r(q, n)$ below three targets, computed by script. The growth is logarithmic in the inverse target risk but explosive in q : a 10% attacker is answered by 6 confirmations, a 45% attacker by 539, and at 50% by no number at all.

Proposition 6.1 (No finite count is final). *For every $0 < q < p$ and every n , $r(q, n) \geq 2q^n > 0$.*

Proof. By Theorem 5.2, $r = 2\Pr[A_n]$, and one way for the attacker to win the first-to- n race is to find the first n blocks outright, an event of probability q^n . $\square$

Proposition 6.2 (Divergence at the majority line). *Fix a target $0 < \varepsilon < 1$. For $0 < q < \frac{1}{2}$, let $n_\varepsilon(q)$ be the least n with $r(q, n) < \varepsilon$. Then $n_\varepsilon(q) \rightarrow \infty$ as $q \uparrow \frac{1}{2}$.*

Proof. First, $n_\varepsilon(q)$ exists. The event A_n that the attacker reaches n blocks first is the event that at least n of the first $2n - 1$ block discoveries are the attacker's. Since $q < p$, each such length- $(2n - 1)$ sequence has probability at most $q^n p^{n-1}$, and there are fewer than 2^{2n-1} sequences. Hence

$$r(q, n) = 2\Pr[A_n] \leq \frac{(4pq)^n}{p} \rightarrow 0,$$

because $4pq < 1$.

For fixed n the function $q \mapsto r(q, n)$ is a polynomial on $[0, \frac{1}{2}]$, hence continuous, and $r(\frac{1}{2}, n) = 1$: at $q = p$ each half of the race of Remark 5.3 has probability exactly $\frac{1}{2}$ by Lemma 3.6 and symmetry. Given any N , for each $1 \leq n \leq N$ there is a $\delta_n > 0$ with $r(q, n) > \varepsilon$ whenever $q > \frac{1}{2} - \delta_n$. Taking $\delta = \min_{1 \leq n \leq N} \delta_n > 0$ excludes every $n \leq N$ throughout that neighbourhood, so $n_\varepsilon(q) > N$. Since N was arbitrary, the claimed divergence follows. $\square$

Remark 6.3 (Nothing is special about six). The paper (§5): “There is nothing special about the default, often-cited figure of 6 confirmations. It was chosen based on the assumption that an attacker is unlikely to amass more than 10% of the hashrate, and that a negligible risk of less than 0.1% is acceptable. Both these figures are arbitrary” — six confirmations are “overkill for casual attackers, and at the same time powerless against more dedicated attackers”. Table 2 is the honest replacement: pick the adversary you defend against and the risk you accept, and read off n ; the pair (q, ε) is a policy, not a law of nature.

6.1 Zcash time: 25-second blocks

Corollary 4.7 says security is purchased in blocks. The wall-clock price is set by the target spacing: 25 seconds in ZIP-218. Since public discoveries arrive at rate p/T_0 , waiting for n confirmations takes

nT_0/p seconds in expectation in this model. Three confirmations cost a nominal 75 seconds and ten cost 250 seconds when all hashpower publishes on the public chain. With a 10% withholding attacker, the expected wait is instead divided by 0.9.

For example, a nominal thirty-minute wait buys 72 confirmations at 25-second spacing, compared with three at Bitcoin’s 600-second spacing. The corresponding strict-overtake probabilities against a 10% attacker are approximately 9.35×10^{-34} and 1.712%, respectively; either count takes $30/0.9 \approx 33.3$ minutes in expectation under withholding. These are model calculations, not guarantees about either network.

The comparison holds q fixed. It contains no network and therefore does not price honest blocks racing one another during propagation. Quantifying that lost work requires a propagation distribution and mining topology. Moreover, a policy waiting for a fixed amount of accumulated miner reward asks a different question: decreasing the reward per block in proportion to the target spacing requires proportionally more blocks to reach that amount. Ignoring transaction fees (value paid for transaction inclusion) and rounding, the shorter spacing then cancels out of the mean wait. The monetary and fixed-hashrate models are not interchangeable.

Remark 6.4 (Where the model bends). Three assumptions do not survive contact with the deployed chain; they determine how cautiously the calculations may be transferred to it. (i) *Difficulty is not constant*: Zcash retunes the target after every block (Section 8.2). The race is scored in work, not block counts, and q may be interpreted approximately as the attacker’s share of work production while the branches’ block weights remain close; once their difficulty schedules diverge, the equal-step random-walk formula is no longer exact. (ii) *Hashrate is not constant*: q in the theorems is whatever share the attacker sustains for the attack’s duration; the paper’s own caveat is that T_0 re-enters only if the attacker cannot sustain his hashrate long enough, which it judges unlikely for a serious attacker. (iii) *Propagation is not instantaneous*: honest self-orphaning wastes honest work and thereby raises the attacker’s effective share; its size is not quantified here. A fourth gap is not the model’s: the attacker of this volume follows the protocol’s validity rules perfectly and attacks only the *choice* between valid histories. Attacks on the rules themselves are other volumes’ subjects.

7 The economics of the attack

The probabilities of Section 5 answer the adversarial merchant, who assumes the customer is an attacker. The paper’s §6 answers the realistic one: when is the attack worth mounting at all? Throughout this section $q < p$ — “[o]therwise, all bets are off” (§6) — and the model is deliberately stylised; its own caveats close the section.

The attacker targets k merchants simultaneously with payments of v each, all invalidated by the same secret branch; the goods are worth αv to him, $0 < \alpha \leq 1$; he gives up after mining o blocks in vain; and each of his blocks would earn the block income B if it ended up in the accepted chain. If the attack succeeds his blocks are all accepted (he keeps goods *and* coins *and* rewards); if it fails he has paid kv , keeps goods worth $k\alpha v$, and his o orphaned blocks forfeit oB . For an exact finite-stop profit calculation, r would have to be the success probability of that stopping policy. Following the paper, the tables below instead insert the unlimited-horizon $r(q, n)$ as a heuristic proxy. They are not an exact evaluation of a strategy that always stops after $o = 20$ private blocks.

Proposition 7.1 (Profitability; Rosenfeld §6). *Given success probability r and these stipulated pay-offs, the attacker’s expected profit over not attacking is*

$$k\alpha v - (1 - r)(kv + oB) = kv(\alpha + r - 1) - (1 - r)oB,$$

positive if and only if

$$v > \frac{(1-r)oB}{k(\alpha+r-1)}$$

(when $\alpha + r > 1$; when $\alpha + r \leq 1$ the modeled attack is never strictly profitable). This modeled attack is not strictly profitable whenever

$$v \leq \frac{o(1-r)B}{k(\alpha+r-1)} \stackrel{\alpha=1}{=} \frac{o}{k} \cdot \frac{B(1-r)}{r}, \quad (3)$$

which with the paper's choices $o = 20$, $k = 5$, $\alpha = 1$, $B = 25$ BTC is its equation (2), $v \leq 100(1/r - 1)$ BTC.

Proof. The gain $k\alpha v$ is certain (the goods are obtained either way); the loss $kv + oB$ is paid exactly when the attack fails, with probability $1 - r$. Rearranging the positivity condition gives the threshold; substituting the paper's constants gives $20 \cdot 25 \cdot (1 - r)/(5r) = 100(1 - r)/r$. $\square$

7.1 Reward-normalised exposure

The *block subsidy* is value issued by the protocol in a block, distinct from fees transferred by its transactions. Its *scheduled* component is base issuance determined by height, before any reserve payout. A *funding stream* is a required allocation of a share of that subsidy to a designated recipient. The *coinbase* is the block's first transaction, which distributes the available subsidy and fees according to the applicable rules. The *reserve* is the accounting balance of funds removed from circulation and awaiting reissuance.

The income B forfeited by an orphaned block is the amount its miner would have retained: miner subsidy plus retained transaction fees, not the subsidy allocated to other recipients. Under NU7 this depends on scheduled issuance, the prior reserve balance, and the applicable fee-removal rule (Section 8.8). A universal ZEC figure would therefore hide assumptions about both chain state and unresolved draft rules.

Instead, Table 3 measures transaction value in units of the chosen block income B . With the paper's $o = 20$, $k = 5$, and $\alpha = 1$, equation (3) becomes

$$\frac{v}{B} \leq 4 \frac{1-r}{r}.$$

Once a particular threat model supplies B , multiplying by it gives the corresponding monetary threshold. Treating B as constant over the attack remains a modelling assumption.

q	$n=1$	2	4	6	8	10
5%	36	271	10,327	344,743	10,931,506	336,737,801
10%	16	67	729	6,759	59,475	508,917
20%	6	15	55	167	467	1,262
30%	2	5	11	21	35	57
40%	1	1	2	4	5	6

Table 3: The largest whole-number multiple of block income B with non-positive heuristic expected profit: $\lfloor 4(1-r)/r \rfloor$, computed by script. The table is dimensionless: it neither fixes a ZEC subsidy nor predicts future fee income. Values inherit every assumption of the model; the caveats of Remark 7.3 apply in full.

Within the model, a handful of confirmations makes the attack unprofitable against small attackers at any plausible payment size; against a 30% attacker, six confirmations make modeled expected profit non-positive only up to about 21 times the assumed block income B . The required count grows only logarithmically in the value at stake, by the geometric decay of Theorem 5.2. These are neither exact finite-stop profit calculations nor bounds on deployed rollback risk; the deployed tie-break, changing difficulty, and rollback rails require separate modelling.

Remark 7.2 (Majority is a different problem). At $q = p$, eventual strict overtake has probability 1, but the walk has zero drift; that fact alone does not give the attacker a lasting monopoly. When $q > p$, the economics change character entirely: the paper notes a strict majority attacker can double-spend at no cost beyond normal mining, reject every other miner’s blocks to take the whole issuance, and exclude transactions at will, driving honest miners out and entrenching himself — so a majority attack “is best seen as an attempt to destroy” the currency, motivated from outside it (a short position, a competing system), not an attempt to profit inside it. Confirmation policy is not the defence at that point; the defence is the cost of assembling the majority, which lies outside this model.

Remark 7.3 (The model’s own caveats). The paper flags each stylisation itself. The give-up point $o = 20$ “is a significant simplification” (an optimal attacker balances completion against compounding losses); the safe values “should be taken with a grain of salt, because of the many modeling assumptions”; and a model resting on mining *costs* rather than forfeited *rewards* would make the required confirmations linear, not logarithmic, in the transaction value — “very poor security, hence in a situation where this is relevant, we have already lost anyway” (§6). The bound’s role is the shape it reveals — logarithmic patience buys exponential safety — not its third significant digit.

8 NU7: 25-second blocks and bounded shielded work

The ZIP-259 draft combines three changes: shorter blocks with limits on shielded work (ZIP-218), reissuance that preserves the halving schedule — periodic divisions of scheduled subsidy by two (ZIP-237) — and rejection of version 4 transactions (ZIP-2003). This section describes that proposal, not a deployed upgrade. Its activation height, denoted A , is unassigned on each network at the source boundary in Section 1.1.

NU7 introduces no new transaction format: versions 5 and 6 remain available. Their signatures nevertheless commit to a *consensus branch identifier*, a value identifying the network upgrade’s rules. An unchanged encoding therefore does not make a pre-NU7 signature valid under NU7. Disabling version 4 also disables spending funds in Sprout, an older shielded pool, because the newer formats cannot represent its transfers; it does not transfer their value to a reserve or remove it from the supply.

8.1 Chain selection and the tie-break

The most-work rule remains Definition 2.2: compare the sum of the blocks’ work, not their count. Changing target spacing changes neither the work definition nor this ordering.

Remark 8.1 (The implementation tie-break). The specification and Rosenfeld’s model prefer the first-seen block at equal cumulative work. The examined Zebra implementation instead uses the larger tip hash. This distinction concerns local chain choice, not the validity of a block. The divergence matters to the attack model. At equal work, an attacker can compare its private tip hash with the public tip and reveal immediately when its hash wins the deployed deterministic tie-break; the strict-overtake boundary $z = -1$ used above does not count that success opportunity. Therefore $r(q, n)$ is exact for the stated strict-overtake model and is a lower bound if only its tie-break is replaced by Zebra’s, keeping equal difficulty, unlimited rollback, and the other assumptions fixed. It is not thereby a lower bound on the full deployed process: changing difficulty and rollback limits also change the success event. Quantifying the extra probability, including any scope to choose among candidate private tips, requires a different stopping rule and is outside this volume.

8.2 Difficulty in motion

Assumption 4.2 freezes difficulty; Zcash retargets after every block. In ZIP-218, the NU7 target spacing is 25 seconds and the target-averaging window is 102 blocks. The timestamps determining the adjustment still come from the branch being evaluated, including on an attacker’s private branch.

Write $N = 102$ for the target-averaging window and $T = N \cdot 25 = 2,550$ seconds for its nominal timespan.

The specification’s §7.7.3 defines the rest of the calculation. For a block height h , let $m(h)$ be the median timestamp of the eleven preceding blocks: their sixth-smallest timestamp. Let $\bar{t}(h)$ be the mean target of the N blocks preceding h : the sum of their targets divided by N . Define the observed and damped timespans by

$$\begin{aligned} a(h) &= m(h) - m(h - N), \\ d(h) &= T + \text{trunc}\left(\frac{a(h) - T}{4}\right), \end{aligned}$$

where trunc discards the fractional part towards zero. The bounded timespan $c(h)$ is $d(h)$ clamped between $[0.84T]$ and $[1.32T]$: values below the lower endpoint are replaced by that endpoint, and similarly above the upper endpoint. The maximum target PoWLimit is $2^{243} - 1$ on Mainnet, the production network, and $2^{251} - 1$ on Testnet, the test network (§5.3). The target calculation is

$$t(h) = \min\left(\text{PoWLimit}, \left\lfloor \frac{\bar{t}(h)}{T} \right\rfloor c(h)\right),$$

followed by the compact encoding defined in the specification’s §7.7.4. The order of the floor and multiplication matters. At NU7 the clamp endpoints are 2,142 and 3,366 seconds; the eleven-timestamp median, fourfold damping, and target cap are unchanged.

Activation does not reset the accumulated work or multiply the target by three in a single operation. The first NU7 blocks still look back over pre-NU7 history and adjust through the same bounded rule. Likewise, the timestamp rules do not shrink with the target spacing: a timestamp must exceed the median of the preceding eleven and be at most 90 minutes above it (§7.6). A further non-consensus receipt check rejects blocks more than two hours ahead of the validator’s clock. Testnet’s exceptional minimum-difficulty rule continues to use six target spacings: its delay threshold becomes $6 \cdot 25 = 150$ seconds. This exception does not apply to Mainnet.

Remark 8.2 (What per-block retargeting changes). The averaging window, damping, and clamp constrain the adjustment but do not prove that two forked branches retain close targets. Work per block moves inversely with the target; a branch’s timestamps can change both its future block rate and the work carried by each block. Equal expected work production does not make the probability of an eventual overtake independent of these jump sizes. Consequently, the probability tables are an equal-work baseline, not an exact analysis of private branches whose difficulty schedules diverge.

8.3 Equihash and the memoryless model

Zcash’s proof of work is Equihash with parameters $n = 200$, $k = 9$ (specification §7.7, §7.7.1; the algorithm is Biryukov and Khovratovich’s, ePrint 2015/946), but the quantity the race counts is gated one step later: “[t]he difficulty filter is unchanged from Bitcoin, and is calculated using SHA-256d on the whole block header (including `solutionSize` and `solution`)”, required to be at most $\text{ToTarget}(n\text{Bits})$ (§7.7.2). Each candidate Equihash solution therefore faces a fresh SHA-256d lottery. Treating distinct headers as independent trials is the usual random-function model for SHA-256d, not a property asserted by the specification; under that model these are the Bernoulli trials of Remark 3.7.

The specification nowhere asserts progress-freeness — the argument is this volume’s, not a citation. What Equihash changes is the granularity: candidates arrive in batches, one batch per memory-hard solver run. Progress within a run is discarded when a competing block arrives. Under independent identically distributed runs, the number of runs until an accepted header is geometric; replacing those discrete completion times by an exponential clock is a further approximation, coarse within one run and good only when a run is short relative to the target spacing: 25 seconds in the NU7 draft. Run duration is implementation- and hardware-dependent and is not quantified here. A

proof-of-work whose solve time were comparable to the spacing would make block discovery visibly non-exponential and pull the attribution sequence away from Proposition 3.4.

8.4 Reorg rails, maturity, and the finality floor

Consensus does not impose a maximum reorganisation depth. ZIP-218 recommends a node-local 600-block rollback window: at the target spacing this is $600 \cdot 25 = 15,000$ seconds, or 250 minutes. Such a limit bounds the history that a node automatically reverses; it is not a proof that older blocks cannot be replaced by a higher-work valid chain. Following a branch outside that retained history requires different local state or operator intervention. Actual node implementations may choose a different window.

Remark 8.3 (What the rails buy). A finite rollback policy truncates some paths to eventual overtake, so the unlimited-rollback probability $r(q, n)$ is not the exact failure probability of a node enforcing that policy. A limit is also not global agreement: different nodes may stop on different sides of a long split. The specification’s §3.3 adds a separate requirement for validators that risk Mainnet funds or display transaction information: their chain must include the activation block of the most recent settled network upgrade. That socially anchored checkpoint is distinct from probabilistic finality.

A *transparent output* records its value and destination in cleartext. The consensus maturity rule prevents spending such an output of a coinbase until the spending block is at least 100 heights later (§7.1.2). ZIP-218 keeps that count, a nominal $100 \cdot 25 = 2,500$ seconds. Coinbase outputs placed directly in a shielded pool do not have this transparent maturity restriction. A 600-block rollback window can therefore reverse an already mature coinbase: maturity permits spending, not finality.

8.5 Wallet confirmation policy

A wallet can distinguish *trusted* outputs that its own transaction handling can recreate after a roll-back from an *untrusted* external payment that may require the sender to pay again. The examined wallet backend uses three confirmations for the former and ten for the latter, with zero-confirmation shielding of transparent funds allowed by its default policy. These choices are not consensus rules.

Table 1 quantifies their modelled trade-off: three confirmations assign a 10% attacker risk 1.712%, while ten assign the same attacker about 0.0008% and a 20% attacker about 0.316%. The count remains a (q, ϵ) policy, not a guarantee.

The *anchor* is the note-commitment-tree root against which a spend proves membership (*Crypto Guide*, §Privacy-preserving membership via zero-knowledge paths). ZIP-218 keeps the recommended anchor depth at three blocks. Its recommended transaction-expiry delta is 120 blocks, a nominal $120 \cdot 25 = 3,000$ seconds, without overriding an explicitly configured expiry delta. There is no instruction to divide every confirmation count by three.

8.6 A block budget for shielded work

A byte limit does not directly bound the computational work inside a block. The 2,000,000-byte limit must therefore be considered alongside the 25-second target spacing and the costs of shielded validation and wallet scanning. ZIP-218 adds limits on the shielded components that drive this work.

An Orchard *Action* pairs one spend slot with one output slot; dummy slots allow either side to carry no real transfer. Ironwood is the newer shielded pool using the same Action structure (ZIP-229). The older Sapling shielded pool counts spends and outputs separately; its cryptographic construction is not needed for this accounting.

Let O and I count Orchard and Ironwood Actions in a block, and let S count Sapling spends plus outputs. With the Ironwood coverage proposed in PR 1361, the complete shielded budget is

$$O \leq 330, \quad I \leq 330, \quad S \leq 300, \quad O + I + S \leq 330.$$

The final inequality is shared across pools: their separate maxima cannot all be filled in one block. Sprout transfers cannot occur because NU7 rejects the version 4 format that represents them.

The Ironwood term remains under review rather than already specified by the merged ZIP-218 draft. For example, $O = 100$, $I = 200$, $S = 30$ exhausts the shared budget; increasing I to 201 violates it even though each per-pool limit still holds. At two Actions per transaction, a block devoted to one Orchard-protocol pool fits at most 165 such transactions under this budget, or a nominal $165/25 = 6.6$ transactions per second. This is a capacity calculation under a specified transaction shape, not a measured throughput guarantee. Transparent components remain subject to the byte limit rather than this shielded-action budget.

8.7 Scheduled issuance with shorter blocks

A *zatoshi* is the atomic currency unit: 1 ZEC equals 10^8 zatoshi. A *halving* divides the scheduled subsidy by two. Retaining its nominal wall-clock interval while tripling block frequency requires both three times as many blocks per interval and one third of the subsidy per block. ZIP-218 therefore changes the full interval from 1,680,000 to $L = 5,040,000$ blocks. These earlier intervals are needed to compute progress at activation, not to supply an alternative rule. The change preserves expected timing, not exact calendar dates, because actual block times remain random.

The progress already made towards the next halving must survive activation. Let H_B be the Blossom activation height: 653,600 on Mainnet and 584,000 on Testnet. Before Blossom, halvings were separated by 840,000 blocks; the initial slow-start schedule, which ramps up the subsidy during the first 20,000 blocks, contributes a shift of 10,000 blocks. These constants and the maximum subsidy $M = 1,250,000,000$ zatoshi are defined in the specification's §5.3 and §7.8. For a height $h \geq A$, the draft's halving count and scheduled subsidy s_h , measured in zatoshi, are

$$k(h) = \left\lfloor \frac{H_B - 10,000}{840,000} + \frac{A - H_B}{1,680,000} + \frac{h - A}{5,040,000} \right\rfloor, \quad (4)$$

$$s_h = \left\lfloor \frac{M}{2 \cdot 3 \cdot 2^{k(h)}} \right\rfloor. \quad (5)$$

The factors 2 and 3 account separately for Blossom's and NU7's spacing changes. If activation occurs during the second-halving era, $k(h) = 2$ gives $s_h = 52,083,333$ zatoshi, or 0.52083333 ZEC, until the next halving; $k(h) = 3$ gives 26,041,666 zatoshi.

The old Mainnet third-halving height $H_3 = 4,406,400$ is therefore not the NU7 third-halving height if $A < H_3$. Converting the remaining interval gives

$$H_3^{\text{NU7}} = A + 3(H_3 - A) = 13,219,200 - 2A.$$

No numerical activation height is assigned here. Nor does retiming the halving function automatically retime a funding stream whose endpoint is specified as a literal block height. The draft does not provide such a change, so the funding-stream endpoints and the rescaled halving heights must be treated separately.

8.8 The NSM reserve and delayed reissuance

The *Network Sustainability Mechanism* (NSM) separates temporarily removed funds from the still-unissued scheduled subsidy. In the merged ZIP-237 draft, the *issued supply* V_h is the total value recorded in the chain's value pools after block h . Each pool is a separately tracked balance for one part of the protocol; tracking a shielded pool's total does not reveal the values of its individual notes. The *NSM reserve* R_h records value removed from circulation and not yet reissued. The reserve is public consensus state, not a spendable pool or an account with a spending key. In particular, it is not 21 million ZEC minus the issued supply: that larger difference also includes future scheduled issuance.

The initial reserve recovers the historical subsidy and fees that miners left unclaimed before NU6. Since ZIP-236, a coinbase must account for its full available value, so that historical shortfall is fixed.

The draft defines the seed R_{A-1} as total scheduled issuance through the last pre-NU6 block minus issued supply there. The exact integer seeds for Mainnet and Testnet remain unspecified.

Let $D \geq A$ be the separately assigned height at which reissuance starts, and let d_h be the non-negative number of zatoshi removed from circulation in block h by whatever removal mechanisms have been deployed. With $L = 5,040,000$ blocks per halving interval, the draft selects

$$f = \frac{\lfloor 6,931,680,000/L \rfloor}{10^{10}} = \frac{1375}{10^{10}}.$$

Its additional subsidy u_h , total block subsidy b_h , and reserve update are

$$u_h = \begin{cases} 0, & h < D, \\ \lfloor fR_{h-1} \rfloor, & h \geq D, \end{cases} \quad (6)$$

$$b_h = s_h + u_h, \quad R_h = R_{h-1} - u_h + d_h. \quad (7)$$

All amounts are integers in zatoshi. Reissuance uses the previous block's reserve, so a removal in the current block cannot finance its own additional subsidy. Before D , removals accumulate without payout. After D , a positive reserve pays at least one zatoshi: because R is an integer and $0 < f < 1$, one has $1 \leq \lfloor fR \rfloor \leq R$ for $R > 0$. The draft rejects a negative reserve; it does not silently clamp it to zero.

For a small exact example, take $R_{h-1} = 100,000,000$ zatoshi (1 ZEC) and $h \geq D$. The additional subsidy is $\lfloor 13.75 \rfloor = 14$ zatoshi. Without a new removal the reserve becomes 99,999,986; with a same-block removal of another 1 ZEC it becomes 199,999,986, but the current payout remains 14. This example does not use the unassigned historical seed.

Why this preserves supply and halvings. Transfers between ordinary value pools cancel in their total. A block increases that total by its subsidy and reduces it by its removals:

$$V_h - V_{h-1} = s_h + u_h - d_h, \quad (V_h + R_h) - (V_{h-1} + R_{h-1}) = s_h.$$

The second identity follows by adding the reserve update; reissuance moves value between the two ledgers without creating a second copy. Scheduled issuance retains its halvings, with the reserve paying an additional amount on top. Funding streams, wherever active, take their prescribed fractions of the *total* subsidy b_h , not only s_h . The resulting subsidy depends on prior chain state as well as height.

To understand the chosen rate, temporarily ignore integer rounding and new removals. The recurrence becomes $R_{h+j} = (1 - f)^j R_h$, so keeping about half the reserve after L blocks requires $f = 1 - 2^{-1/L} \approx (\ln 2)/L$. The draft implements an integer-scaled approximation with numerator 1375. Indeed, $(1 - 1375/10^{10})^{5,040,000} \approx 0.50007$. This is a half-life of the reserve without new contributions, not a new halving rule for the scheduled subsidy; small reserves depart from the smooth curve because every positive payout rounds up.

Unresolved deployment and fee details. ZIP-259 selects a reissuance date corresponding to February 2031, but the network-specific height D remains unassigned. The drafts do not yet specify a complete, consistent NU7 removal rule: ZIP-235 requires removal of at least 60% of fees through an explicit coinbase amount, while ZIP-259 describes a block-level contribution requiring no new transaction data. The reserve equations therefore leave d_h as a defined input. Neither a particular fee-removal mechanism nor an exact NU7 miner income is inferred from these incomplete rules.

9 Layer boundary

Every theorem above rests on the *Math Guide's* probability section: the definition of probability, its extension beyond finite spaces, and the continuity theorem used in the catch-up proof. This volume supplies the best-chain model and the quantitative bounds in Tables 1, 2, and 3. Policies that decide which chain view to trust or when to treat a transaction as final lie above that boundary.